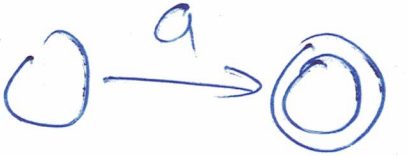
Cpts 317. 9/30

Today: language-to-automaton construction

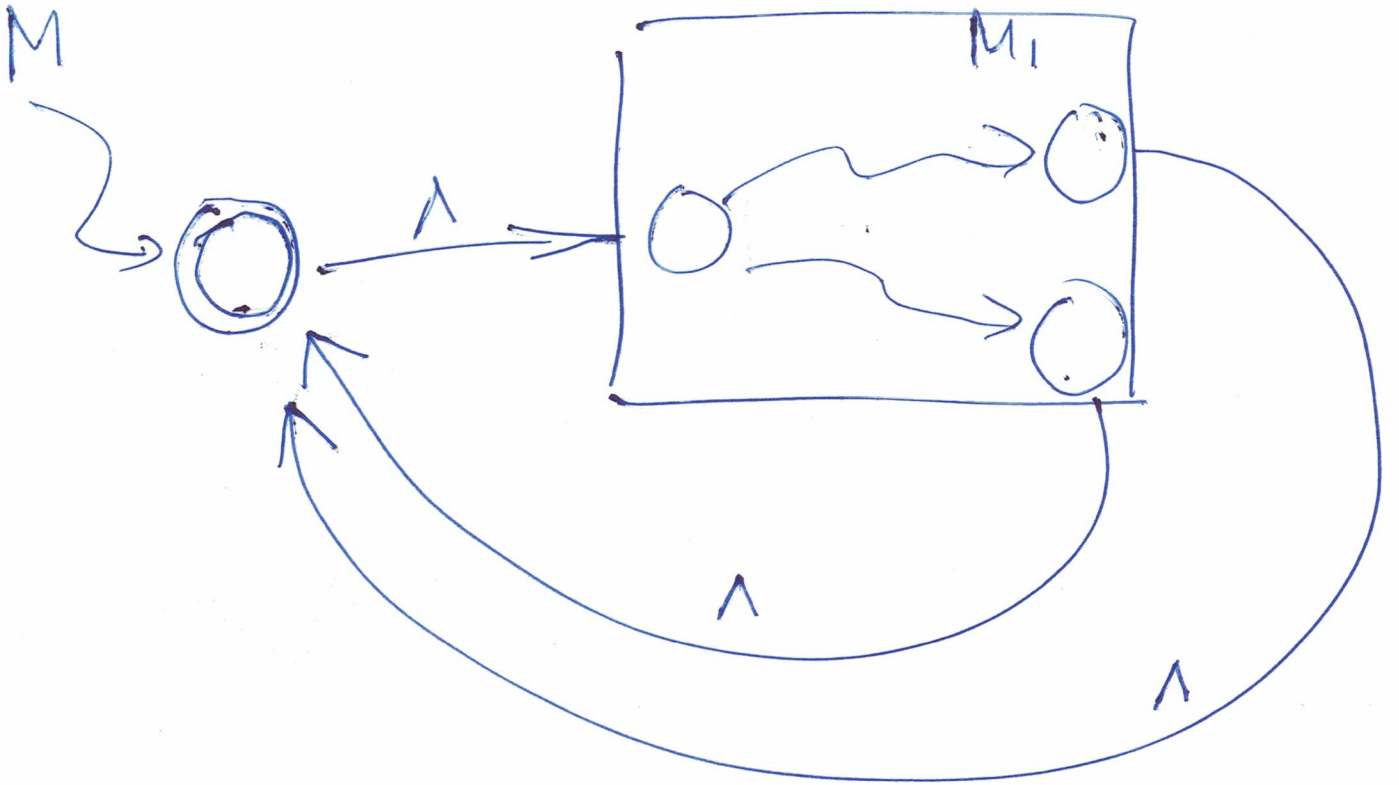
1. $L = \emptyset$,

2. $L = \Lambda$,

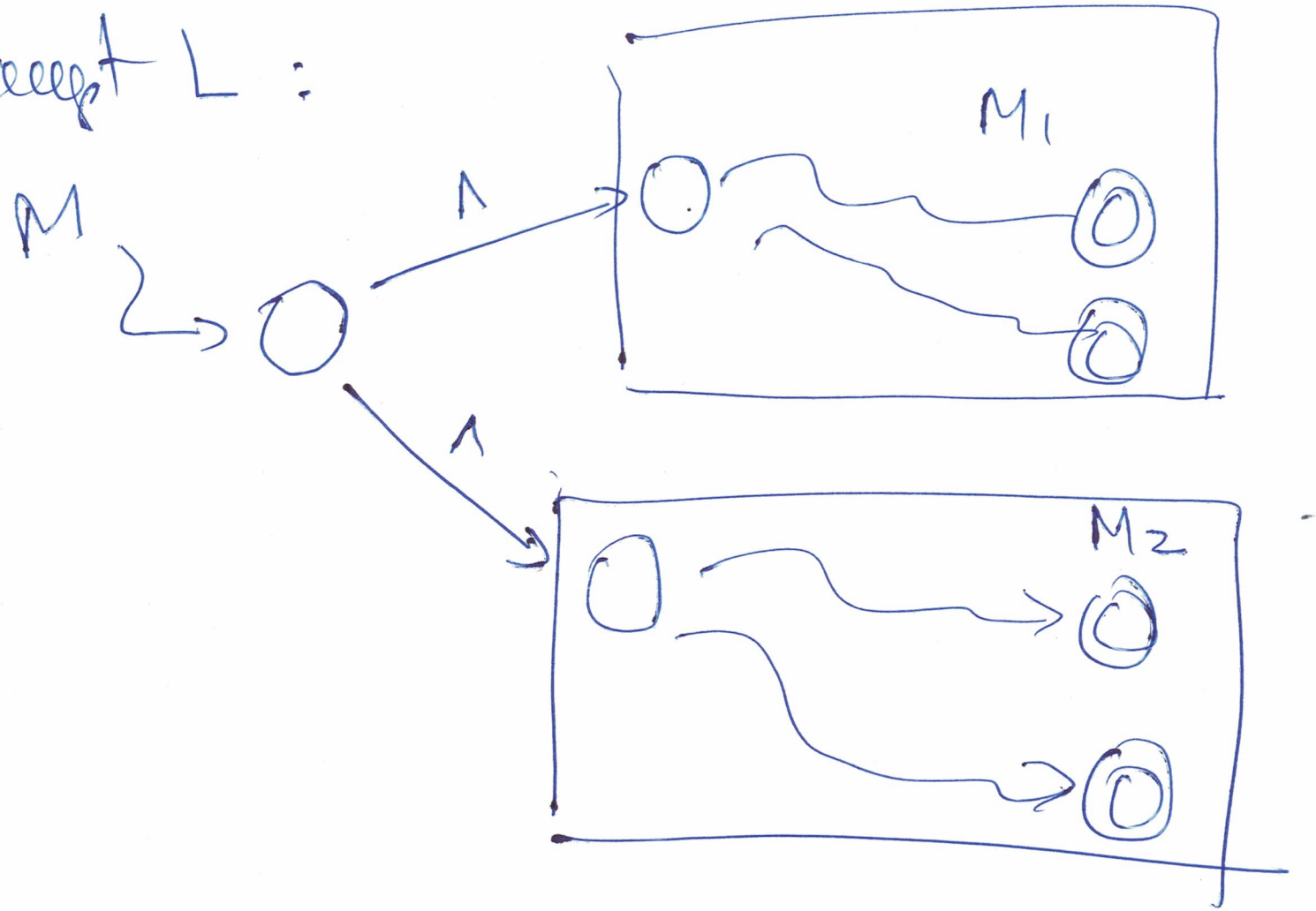
3. $L = a$, $a \in \Sigma$, 

4. Let L_1 and L_2 be two reg languages & M_1 and M_2 be two FA accepting L_1 and L_2, resp.

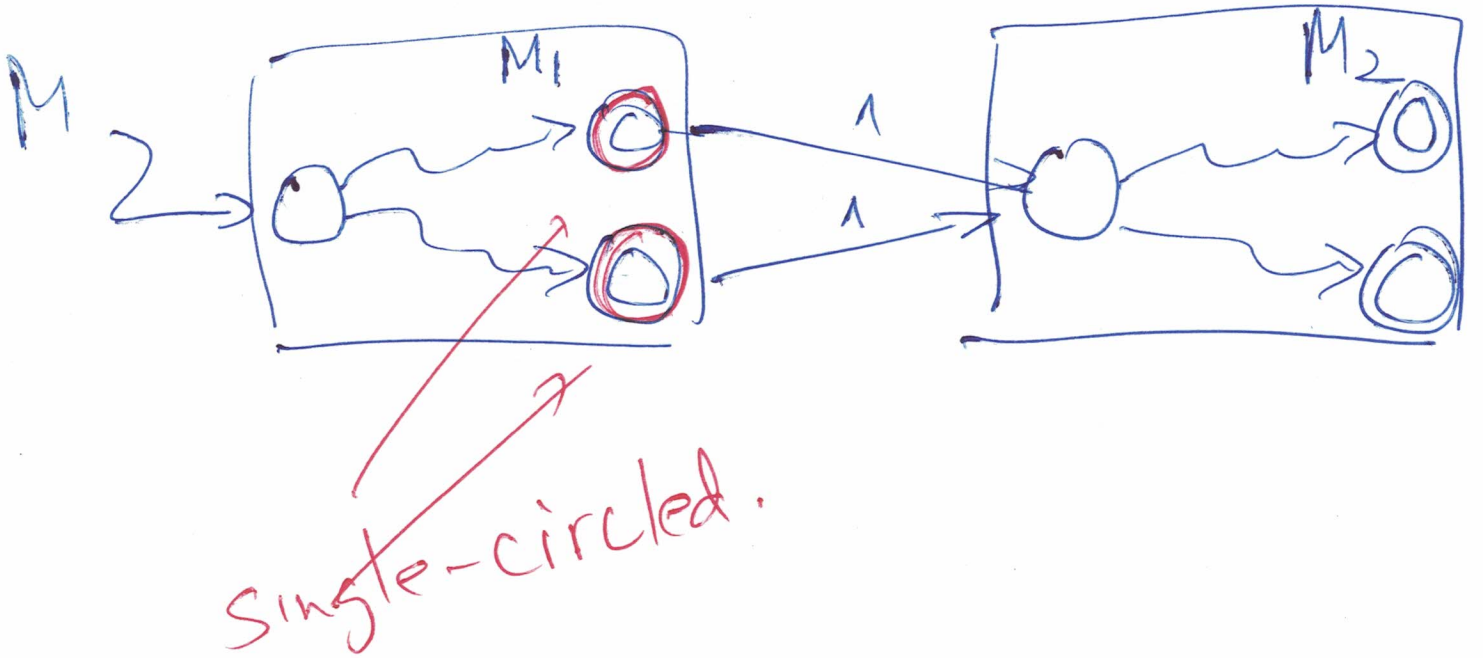
4.3. $L = L_1^*$. I can draw FA M to accept L:



4.1. $L = L_1 + L_2$. I draw FA M to accept L:



4.2. $L = L_1 \cdot L_2$. I can draw FA M to accept L:

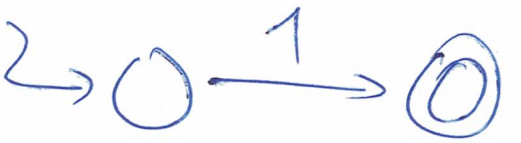


Use Kleene thm to draw/construct a FA from the following regExp:

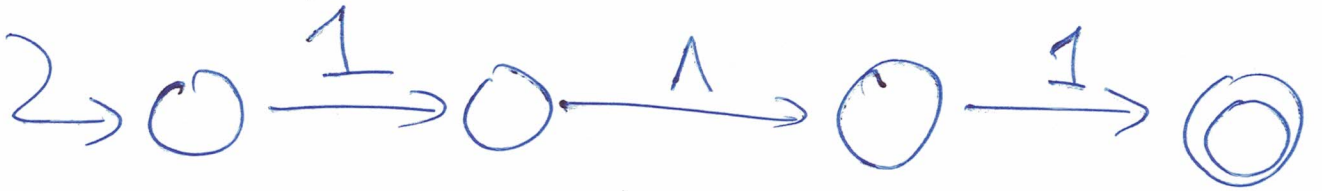
$L = 1.1.$

M_1

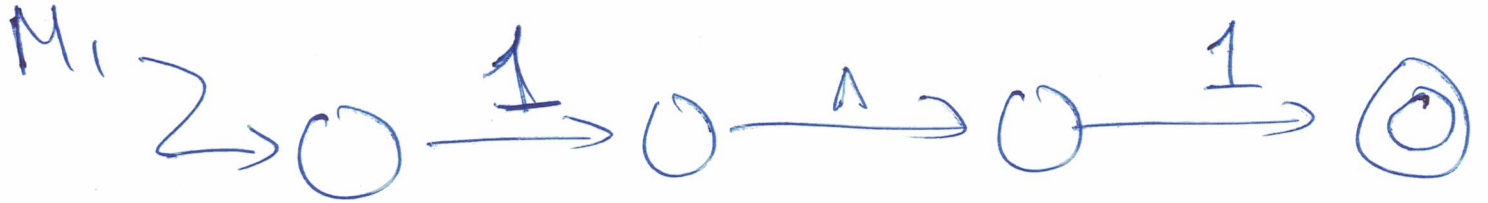
M_2

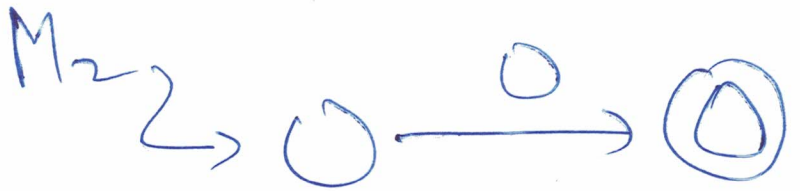


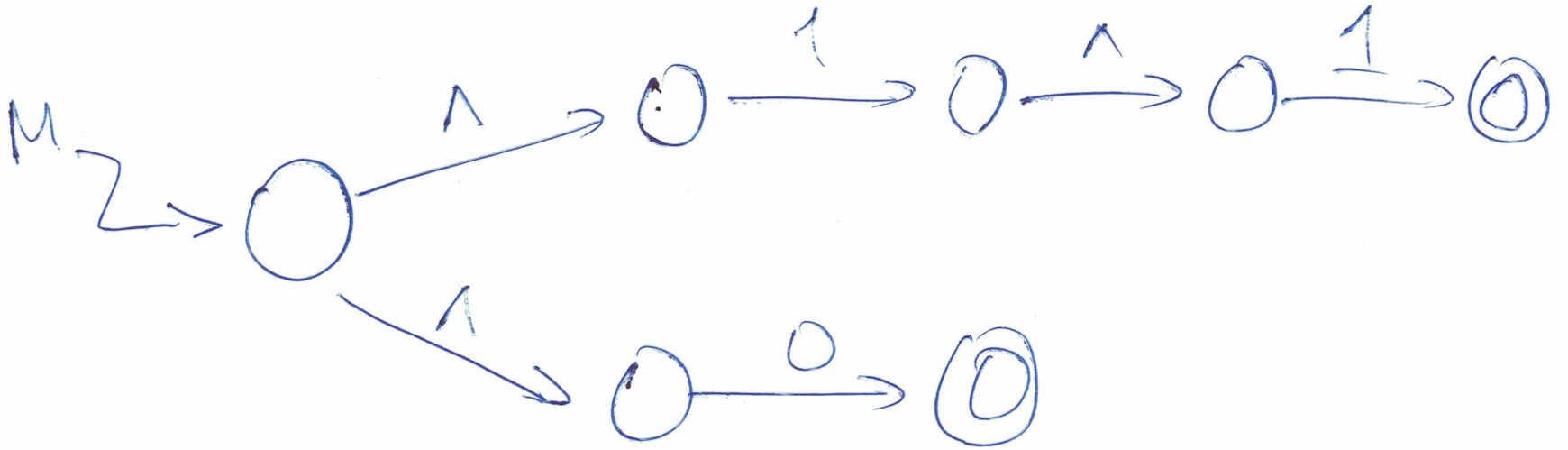
M



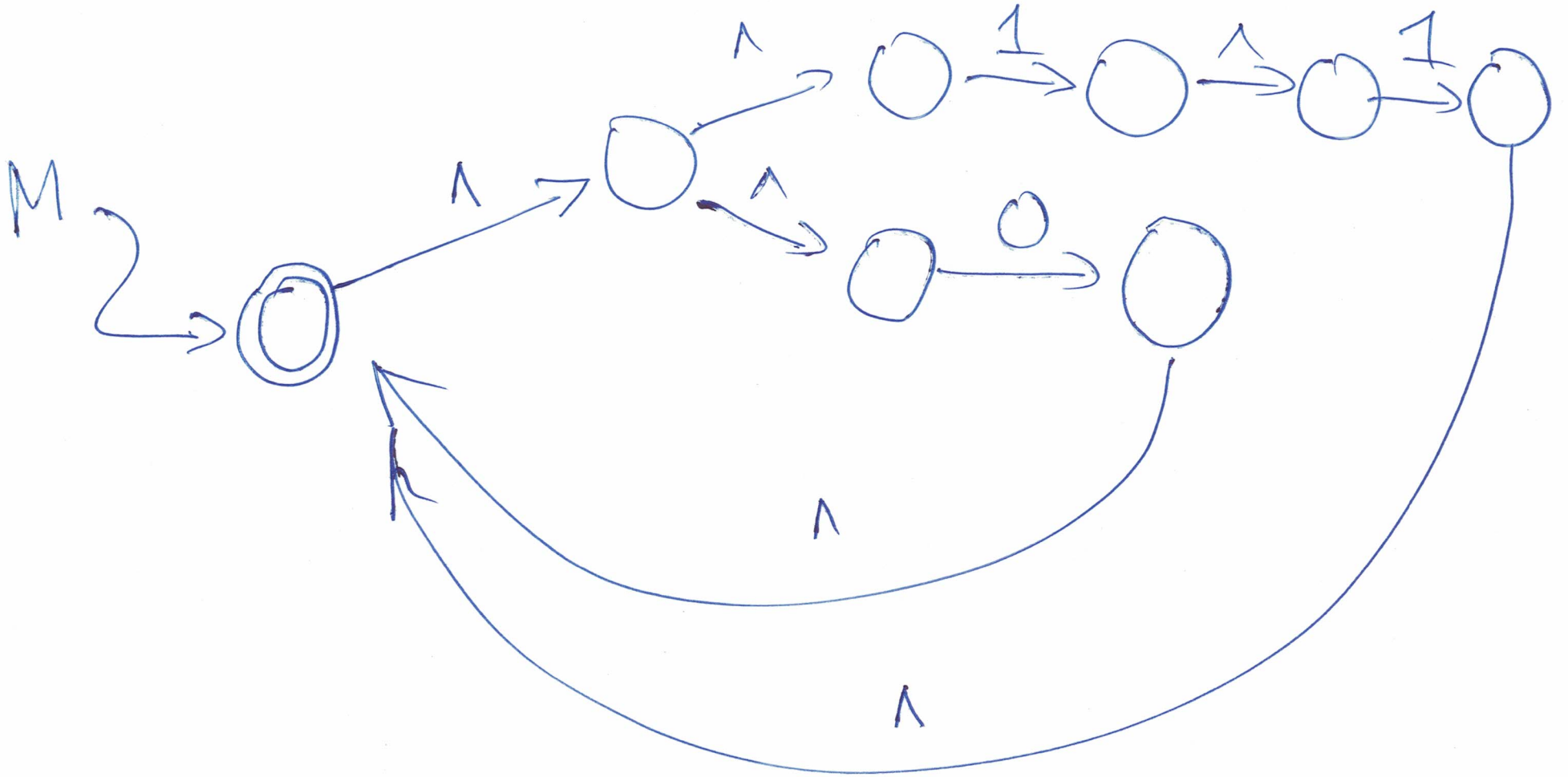
$L = 11 + 0$



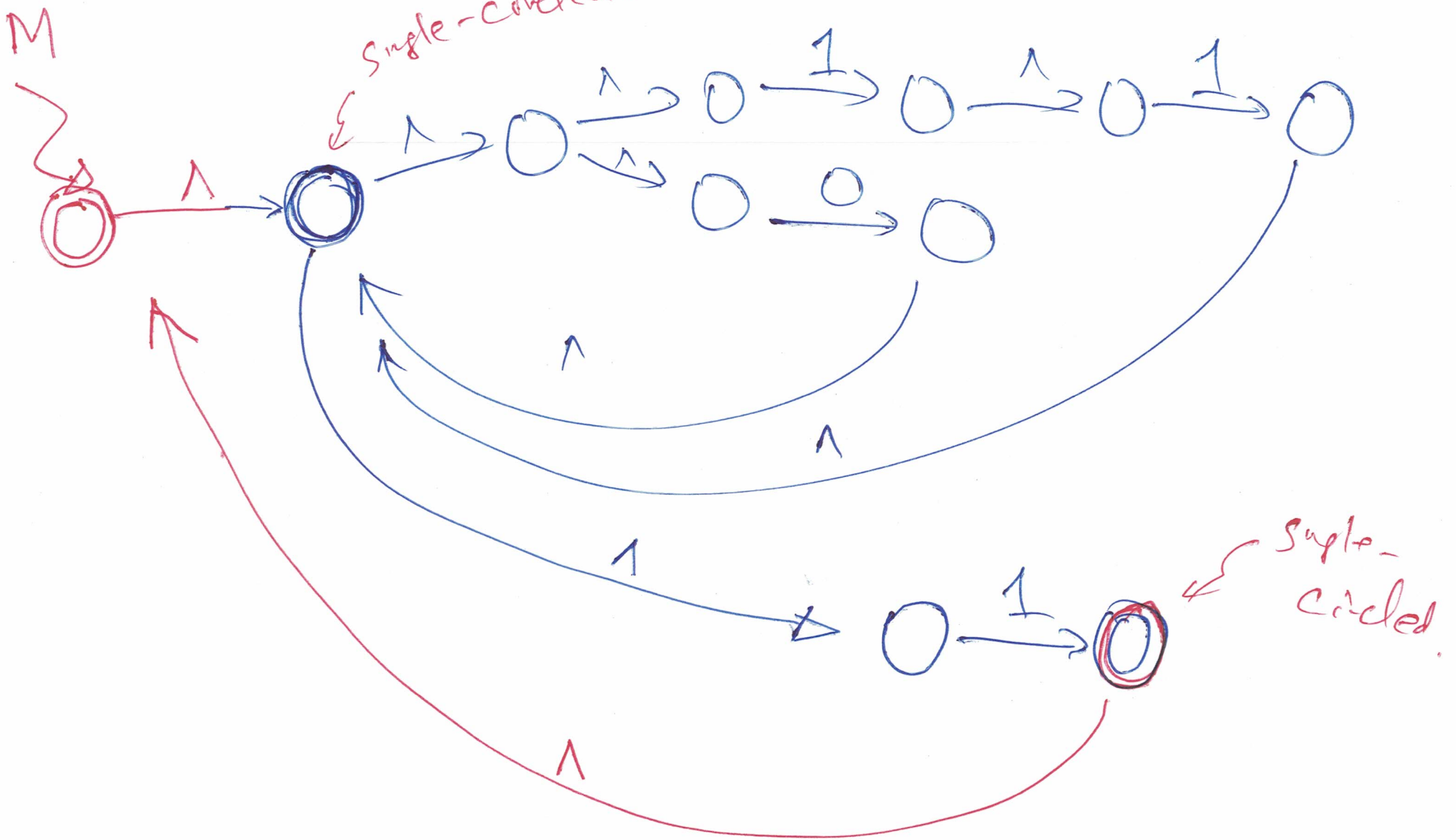




$$L = (11+0)^*$$


M
Λ
Λ
Λ
1
Λ
1
Λ
0
Λ
Λ

$$L = \left((11+0)^* 1 \right)^* .$$



Summary:

efficient

RegExp $\Longrightarrow$ Λ-NFA

($\Longrightarrow$ NFA $\Longrightarrow$ DFA)

Λ-elimination efficient

subset construction exp. blow-up.

Automaton-to-language construction.

Let M be a FA, where we have n states.

WLOG, we can number the states with

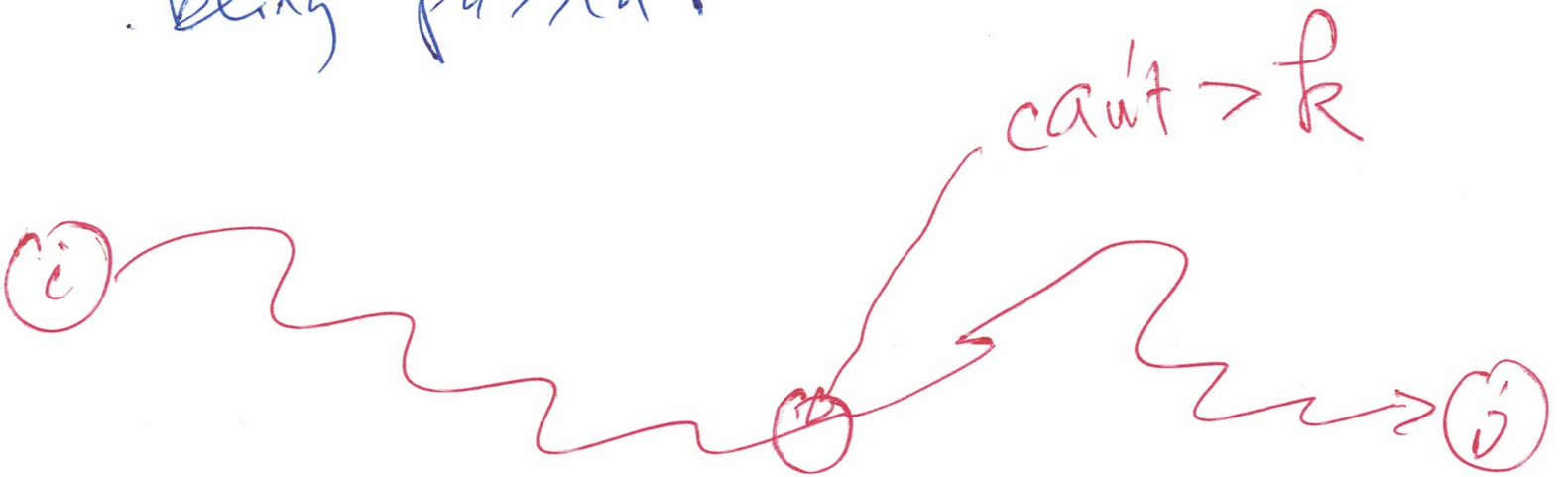
$$1, 2, 3, \ldots, n$$

initial

I want: regExp for $L(M)$?

I define

$R_{ij}^{k} :=$ the set of all strings on a walk from i to j where in between (excluding two ends i & j) there is no node higher than k being passed.

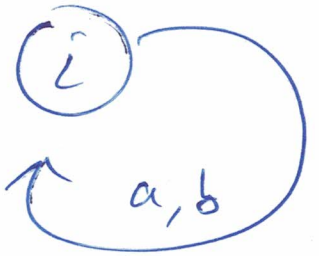


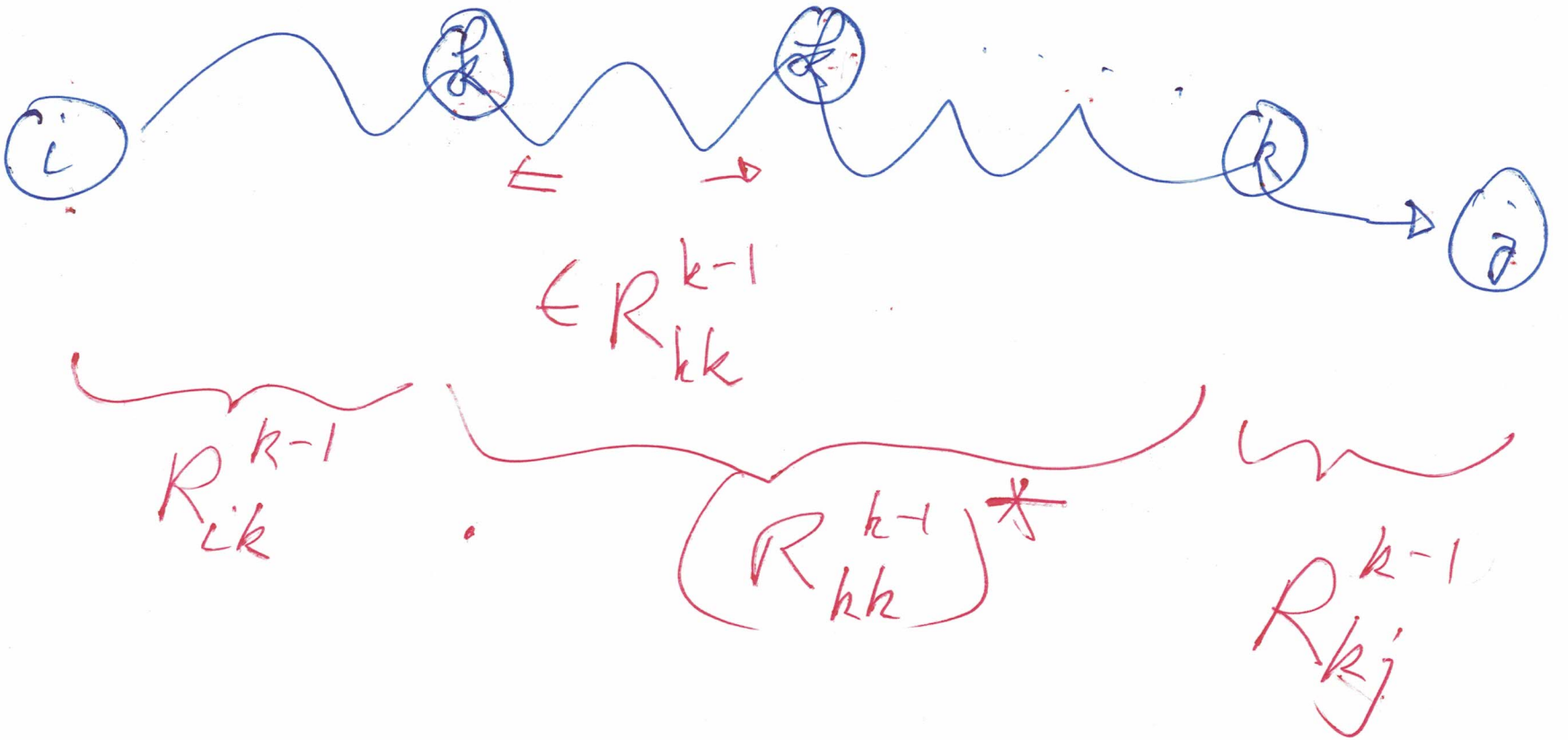
Recall: $R^{n}_{1n} = L(M)$.

initial accepting

$$R^{0}_{ij} = \begin{cases} a_1 + \dots + a_l & \text{if } i \xrightarrow{a_1, \dots, a_l} j \\ \emptyset & \text{o.w.} \quad i \neq j \end{cases}$$

$$i \xrightarrow{a,b} j$$



$$R_{ii}^{0} = \begin{cases} a_1 + \dots a_\ell + \Lambda & \text{if } i \xrightarrow{a_1 \dots a_\ell} i \\ \Lambda & \text{o.w.} \end{cases}$$

$$R_{ij}^{k} = R_{ik}^{k-1} \left(R_{kk}^{k-1}\right)^{*} R_{kj}^{k-1} + R_{ij}^{k-1}$$